Vico – Türen & Tore

Wartungs- und Mängeldokumentation für Türen und Tore. Stand: März 2025.

Inhaltsverzeichnis

1. Quick Start
2. Architektur
3. Features
4. Datenbank
5. Deployment
6. Projektstand
7. Roadmap
8. Projektstruktur

---

1. Quick Start

Tech Stack

Bereich	Technologie
Web-App	Vite + React + TypeScript (TailwindCSS)
Backend	Supabase (Auth, Database, Storage)
Hosting	Netlify

Befehle

Ziel	Befehl
Web Dev	`npm run dev`
Web Dev (Netzwerk)	`npm run dev -- --host`
Web Build	`npm run build`
Tests	`npm run test` / `npm run test:run`
Lint	`npm run lint`
Vico-Dokumentation als PDF	`npm run generate-vico-pdf`

---

2. Architektur

Datenmodell

```
Kunde → BV → Objekt → Wartungsprotokolle
```

Routen

Pfad	Komponente

`/`	Startseite (Dashboard)
`/kunden`	Kunden
`/kunden/:id/bvs`	BVs
`/kunden/:id/bvs/:bvId/objekte`	Objekte
`/kunden/.../objekte/:objectId/wartung`	Wartungsprotokolle
`/suche`	Suche
`/auftrag`	Auftrag anlegen
`/scan`	QR-Scan
`/historie`	Historie (Admin)
`/einstellungen`	Einstellungen
`/benutzerverwaltung`	Benutzerverwaltung (Admin)
`/profil`	Profil

Rollen

Rolle	Rechte
Admin	Vollzugriff, Benutzerverwaltung, Historie
Mitarbeiter	CRUD Stammdaten + Aufträge (außer Löschen von Kunden, BVs, Objekten; BV anlegen nur Admin)
Operator	Nur Wartungsprotokolle schreiben, Stammdaten/Aufträge lesen
Leser	Nur lesen
Demo	Wie Mitarbeiter, aber nur eigene Daten; keine Auftragszuweisung; Daten nach 24h gelöscht
Kunde	Portal-Benutzer (extern): Nur Wartungsprotokolle der eigenen Kunden lesen + PDF-Download

---

3. Features

Kunden & BVs

- Adressfelder (PLZ, Ort, Straße, Hausnummer)
- BVs unter Kunden ausklappbar
- Objekte unter BVs ausklappbar

Objekte

- Stammdaten, Art, Technik, Schließmittel, Feststellanlage, Rauchmelder
- Fotos, QR-Code (Druck via Bluetooth)

Wartungsprotokolle

- Prüfgrund, Herstellerwartung, Feststellanlage, Rauchmelder
- Mängel, Fotos, Unterschriften
- PDF-Export, E-Mail-Versand

Offline & Sync

- Lokale Speicherung, Outbox
- Sync-Status: Offline, Ready, Synchronisiert

Offline-fähige Funktionen

Bereich	Lesen	Schreiben
Kunden, BVs, Objekte, Aufträge	✅	✅ CRUD
Objekt-Fotos, Wartungsprotokolle, Wartungsprotokoll-Fotos	✅	✅
Wartungserinnerungen	✅	—
Komponenten-Einstellungen	✅	⚠️ nur lokal
Historie, Benutzerverwaltung, PDF/E-Mail	❌	❌

Potenziell offline-fähig

Komponenten-Einstellungen (Outbox), Historie (Cache), Profil, Benutzerverwaltung Lesen, PDF-Export (Fotos aus Cache), E-Mail-Outbox, Lizenz-Status (Cache).

---

4. Datenbank (Supabase)

Tabellen

- profiles, customers, bvs, objects
- object_photos, maintenance_reports, maintenance_report_photos, maintenance_report_smoke_detectors
- orders, component_settings, audit_log, license
- customer_portal_users (Kundenportal)

Schema

`supabase-complete.sql` im Supabase SQL Editor ausführen (idempotent). Enthält Rollen, RLS, RPCs, Audit-Trigger, Kundenportal-Tabellen.

---

5. Deployment (Netlify)

1. **Git:** Repo mit GitHub verbinden
2. **Netlify:** Add site → Deploy with GitHub
3. **Build:** `npm run build`, Publish: `dist`
4. **Env:** `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`
5. **Supabase:** Site URL + Redirect URLs (`/reset-password`)

Lizenz-Admin (`admin/`): Separates Netlify-Site, Root: `admin/`, Subdomain z.B. `admin.vico-tueren.de`.

Supabase Keep-Alive (Free-Tier)

GitHub Actions `.github/workflows/supabase-keepalive.yml` – Mo + Do 9:00 UTC. Secrets: `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`.

Demo-Account (24h-Löschung)

Rolle "demo", RPC `cleanup_demo_customers_older_than_24h()`, GitHub Actions täglich 4:00 UTC.

---

6. Projektstand

Implementiert

- versionUtils, UpdateBanner, objectUtils
- Objekt-Anzeige (Suche, Auftrag, Wartung, PDF, QR, Wartungsstatus-Ampel)
- Rechte (Admin, Demo-Rolle, BV nur Admin)
- UX: Dunkelmodus, LoadingSpinner, Toast, Touch-Targets
- Web-App-Test-Checkliste, Historie (Audit-Log)
- Adressuche (OpenPLZ), Fehlerbehandlung (ToastContext)
- Types, Indizes, Code-Splitting
- Unit-Tests, ESLint, CI/CD, PWA

---

7. Roadmap

Roadmap: Geplante Features (mit Aufwand)

#	Feature	Priorität	Aufwand	Beschreibung
1	Wartungsplanung / Erinnerungen	Hoch	3–5 T	Erinnerungen (z.B. 30 Tage vorher), optional E-Mail an Techniker/Kunden
2	Wartungsstatistik / Auswertung	Hoch	3–4 T	Wartungen pro Kunde/BV/Objekt, Auslastung, überfällige Wartungen
3	Export für Buchhaltung	Hoch	2–3 T	CSV/Excel-Export für Abrechnung
4	Schnellzugriff / Zuletzt bearbeitet	Hoch	1–2 T	Zuletzt bearbeitete Kunden/Objekte auf Startseite
5	Erweiterte Filter	Hoch	2 T	Filter Kundenliste (PLZ, Wartungsstatus, BV-Anzahl)
6	Umbau Wartung (MVP)	Hoch	15–20 T	Auftrag → Monteursbericht → Freigabe → Portal (Phasen 1–4)
7	Arbeitszeiterfassung (Modul)	Mittel	5–8 T	Lizenzmodul: Start/Ende, Pausen, ArbZG, Outbox
8	Wartungs-Checkliste pro Objekttyp	Mittel	3–4 T	Vordefinierte Checklisten je Tür-/Tortyp
9	Mängel-Follow-up	Mittel	3 T	Offene Mängel tracken, Status offen/behoben
10	Kalender-Sync (iCal)	Mittel	2–3 T	Aufträge als iCal/Google-Kalender
11	2FA	Mittel	2–3 T	Zwei-Faktor-Authentifizierung für Admins
12	Offline-Erweiterungen	Mittel	3–5 T	Komponenten-Einstellungen Outbox, Historie-Cache, PDF offline
13	Dokumente/Anhänge pro Objekt	Niedrig	4 T	Zeichnungen, Zertifikate pro Objekt
14	Bulk-Operationen	Niedrig	3 T	Mehrere Objekte/Kunden gleichzeitig bearbeiten

15	Portal: Push-Benachrichtigungen	Niedrig	2–3 T	Kunden bei neuem Wartungsbericht informieren

Aufwand: T = Tage (geschätzt, 1 Entwickler)

Umbau Wartung: Auftrag → Monteursbericht (Detail)

Ziel: Detaillierte Aufträge, Abarbeitung durch Monteur, Monteursbericht (Zeiten, Material), Freigabe → Kundenportal + Buchhaltung.

Auftragstypen: Einbau, Reparatur, Wartung, Nachprüfung, Sonstiges. Bei Wartung: Wartungsprotokoll nach DIN 14677 (Feststellanlagen) und ASR A1.7 (Türen/Tore).

Phasen: (1) Aufträge erweitern → (2) Monteursbericht (Tabellen, CRUD) → (3) Freigabe-Workflow → (4) Kundenportal → (5) Wartungsprotokoll DIN/ASR → (6) Buchhaltungs-Export → (7) Offline, Material-Stammdaten → (8) Erweiterungen (Mehrfachbesuche, Vorlagen).

Arbeitszeiterfassung (Modul)

Aktivierung: Lizenzmodul über admin/. **Ablauf:** Start/Ende Arbeitszeit, Pausen (manuell oder automatisch nach ArbZG § 4). **Technisch:** Tabelle `time_entries`, Feature-Flag, Route `/arbeitszeit`, RLS.

---

8. Projektstruktur

```
Vico/
├── src/              # Web-App (Vite + React)
├── portal/           # Kundenportal (separates Vite-Projekt)
├── admin/            # Lizenz-Admin (separates Vite-Projekt)
├── public/           # Favicon, Logo, Checkliste-PDF, Vico-Dokumentation.pdf
├── scripts/          # generate-checklist-webapp-pdf.mjs, generate-vico-pdf.mjs
├── supabase/         # Edge Functions
├── supabase-complete.sql
├── Vico.md           # Diese Dokumentation
└── BENUTZERANLEITUNG.md
```